Understand the project

Start here: the problem, inputs, and formal AI formulation

One-sentence problem: Given a departure time, two campus locations, and any closed railway tunnels, find the connected shuttle route with the least modeled travel time. The map is fictional and reproducible; it is a test network, not VIT road data.

WHAT HAPPENS WHEN YOU PRESS RUN A* SEARCH

The browser sends the selected inputs and map seed to **app.py**. **sample_map.py** generates 14 locations and 17 roads. **routing.py** performs A* and Dijkstra. The server returns the route, road costs, and expansion trace. The browser displays the evidence and animates the chosen route. The 3D renderer never chooses a route.

SEARCH FORMULATION (MEMORIZE THIS)

State	Current road-graph node and its arrival time.
Initial state	Chosen start location at the selected departure time.
Action	Travel to a directly connected node on an open road.
Transition	Move to that node; advance the clock by the edge travel time.
Goal test	Current node equals the selected destination.
Path cost	Sum of travel seconds over the traversed roads.

Railway rule: The railway divides north and south. The only graph edges crossing it are the west and east road tunnels. Closing a tunnel removes that edge from search. If both close, trips across the railway have no path; trips on one side can still work.

Road cost: Each road has a free-flow time and morning/evening congestion multipliers. While the shuttle is on a road, progress is integrated across a time-period boundary, so a road entered at 09:59 can finish after the 10:00 peak ends. A later start cannot arrive earlier on the same edge merely because a period changed (FIFO property).

Scope: We optimize in-vehicle road time only. Stops, roads, traffic and delay spread are generated simulation values. There is no real shuttle timetable, passenger waiting time, live traffic, capacity, or surveyed campus geometry.

Explain the AI core

A*, the heuristic, and a worked search on the default map

THE DECISION RULE

A* keeps candidate locations in a priority queue ordered by **f(n) = g(n) + h(n)**. **g** is actual modeled travel time from the start to n. **h** is an optimistic estimate of remaining time. The algorithm pops the smallest f, relaxes open neighboring roads, saves a parent when it finds a cheaper arrival, and stops when it pops the goal. The dashboard shows each road decision and the resulting queue. Following parents backwards reconstructs the route.

Our heuristic is **straight-line distance from n to goal / 15 km/h**, converted to seconds. Example: 0.50 km implies at least 2 minutes. Every modeled road is at least as long as straight-line distance, free-flow speed never exceeds 15 km/h, and congestion multipliers are at least 1. Therefore h cannot overestimate the remaining road time. The same triangle/edge bound makes it consistent; after a node is popped, its best arrival is final under this FIFO road model.

PSEUDOCODE IN PLAIN WORDS

1. Put start in the min-heap with g=0 and f=h(start).
2. Pop the lowest-f entry; ignore stale entries replaced by a better g.
3. If it is the goal, follow parent pointers and return the route.
4. For each open outgoing road, compute arrival-dependent edge time.
5. If new g is smaller, save parent and push neighbor with f=new g+h.
6. If the heap empties, report no route and retain the exploration trace.

REAL RUN: SEED 17, A TO D, 12:00

The first four A* expansions below are read from the running program (minutes):

Expanded node	g	h	f
A	0.00	3.78	3.78
B	0.96	3.64	4.60
E	2.22	3.53	5.75
C	3.16	2.74	5.90

Final route: **A - B - C - D**. Its three road times add to **8.30 minutes**. A* expanded 11 nodes; Dijkstra expanded 14 and returned the same minimum time.

Why Dijkstra? It is the same search engine with h=0, so it is a comparison baseline for route cost. Agreement is a consistency check, not independent proof. A* may explore fewer nodes because h guides it, but that is not guaranteed on every input. Tiny runtime measurements are noisy; compare exact cost and expansion counts first.

Defend the demonstration

Observed scenarios, tests, assumptions, and honest limits

THREE INPUTS THE PROFESSOR CAN CHANGE

12:00, open	west tunnel; 8.30 min; 11 A* expansions
08:30, open	east tunnel; 12.46 min; 11 A* expansions
12:00, west shut	east tunnel; 9.17 min; 10 A* expansions

At 12:00 the shorter west crossing wins. In the modeled morning peak, its congested approach loses to the east crossing. Closing west forces the east route. Closing both makes a north/south trip unreachable; the UI still shows the nodes explored before the frontier emptied.

WHAT WAS ACTUALLY CHECKED

The automated suite compares A* and Dijkstra for **43,120** combinations: 5 map seeds x 11 times x 4 tunnel states x 14 starts x 14 goals. It asserts identical reachability and equal optimum cost. Separate tests check invalid inputs, same start/goal, a disconnected graph, traffic-period boundary behavior, and the default scenario switches. The browser check exercises controls, trace, source view, alternate seed, and the no-route case.

THE OPTIONAL RELIABILITY EXPERIMENT

For this small map, we enumerate simple paths and rank them by **estimated P90 = mean route time + 1.28155 x sqrt(sum of edge variance)**. This assumes independent, roughly normal road-delay variation. It is a separate experimental objective, not the A* heuristic and not a real measured 90th percentile. For the default trip, fastest west is 8.30 min with modeled P90 12.71; the east reliability route is 9.17 min with modeled P90 10.52.

WHAT WE CANNOT CLAIM

We cannot claim real VIT routes, actual shuttle ETAs, measured uncertainty, calibrated peak hours, or real stop positions. The seed changes coordinates and road parameters while preserving the same 14-node, two-tunnel topology; it does not generate arbitrary graph structures. A* is optimal for the stated model, not for unmodeled waiting, capacity, or driving restrictions. Enumerating every simple route for reliability becomes exponential on a large map.

AI-use declaration for the final report: State that an AI assistant helped draft/refine the synthetic data, code, tests, UI, and study guide. The submitted program runs its own A* and Dijkstra locally; no external AI service decides routes. Every team member should inspect the code and verify the explanation independently.

Viva quick answers

Short answers to likely follow-up questions

Where does the AI make its decision?

In routing.py, _search pops the smallest f=g+h from a heap and updates a neighbor only when a cheaper arrival is found.

Why is this A* rather than ordinary shortest path?

The priority includes h, an informed lower bound to the goal. Dijkstra is the same search with h=0.

Why is your heuristic safe?

Straight-line distance cannot exceed a road path's distance; no edge exceeds 15 km/h and congestion never speeds it up.

Could A* return a wrong optimum?

Yes if the heuristic overestimates, road costs violate the model assumptions, or we add non-FIFO behavior without changing the search state.

Why does arrival time matter?

A road's congestion depends on when the shuttle reaches it, not just the trip's original departure time.

What if a road crosses 10:00?

edge_travel_seconds integrates progress before and after 10:00 instead of charging the whole edge at one rate.

Why can both closed tunnels yield no route?

The railway separates north and south; those are the only crossing edges. Search exhausts the reachable side.

What if start equals destination?

The first popped node is already the goal: path contains one node, cost is zero.

Why compare with Dijkstra?

Both should return the same minimum modeled time. Their expansion counts show how much the heuristic helps on each case.

Why might A* not expand far fewer nodes?

Straight-line time is a weak lower bound when road detours or congestion dominate, so its priorities resemble Dijkstra's.

Why use a priority queue?

It quickly selects the next lowest-f candidate; a plain list would repeatedly scan all waiting nodes.

What does relaxing a road mean?

Compute a possible arrival at its neighbor. If it beats the saved arrival, update that g value and parent.

What is the rough complexity?

With a consistent heuristic and a binary heap, graph search is roughly O(E log E) time and O(V+E) space for fixed-time edge evaluation. The optional all-path reliability mode is exponential.

Is the reliability path guaranteed reliable in reality?

No. Its P90 is conditional on synthetic, independent normal-delay assumptions, not observed traffic.

What would you improve with real data?

Survey legal shuttle roads and stops, measure time by period, calibrate delays, then validate predicted ETAs against recorded trips.

RUN IT

Browser: python app.py, then open http://127.0.0.1:8000
Code-first: python review_demo.py --time 08:30 --seed 17
Tests: python -m unittest -v